An Investigation into Action Space Constraints in Reinforcement Learning

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **15**
1.1 Opening Statement 15
1.2 Aims of the Project 15
1.3 Objectives 15
1.4 Project Motivations 16
1.5 Risks 16

2 Literature Review **17**
2.1 Chess fundamentals 17
2.1.1 Game rules 17
2.1.2 Board representation 17
2.1.3 Move generation 17
2.2 Machine learning in chess 17
2.2.1 Policy 17
2.2.2 Value 17
2.2.3 MCTS 17
2.3 History 17
2.3.1 Alpha beta pruning 17
2.3.2 NNUE 17
2.3.3 Transformers (attention is all you need) 17
2.4 Now 17
2.4.1 Grokking (a classic) 17
2.4.2 AlphaZero (Masking) 17
2.4.3 MuZero (Latent space) 17
2.4.4 leela chess zero 17
2.4.5 Chess transformer 18
2.4.6 Chessformer 18

3 Research Hypotheses **19**
3.1 Pseudo legal training 19
3.2 Punishment propogation 19
3.3 autoregressive move generator 19
3.4 Perfect data injection 19

4 Implementation details **20**

4.1 System architecture overview . . . 21
4.2 Client Architecture . . . 21
4.2.1 Bitboard . . . 21
4.2.2 Chessboard . . . 21
4.2.3 ChessSquare . . . 21
4.2.4 CastlingRights . . . 21
4.2.5 ChessMove . . . 21
4.2.6 ChessSquare . . . 21
4.2.7 ChessGame . . . 21
4.3 Model Architecture . . . 21
4.3.1 Encoder . . . 21
4.3.2 Positional Embeddings . . . 21
4.3.3 Policy . . . 21
4.3.4 Value . . . 21
4.4 MCTS Architecture . . . 21
4.4.1 Data . . . 21
4.4.2 PUCT algorithm . . . 21
4.4.3 Node . . . 21
4.4.4 Edge . . . 21

5 Training & Self Play . . . 22
5.1 Self play . . . 22
5.2 MCTS . . . 22
5.3 Training (backprop via autograd) . . . 22

6 Results . . . 23
6.1 Hypotheses 1: pseudo legal . . . 23
6.2 Hypotheses 2: masking . . . 23
6.3 Hypotheses 2: EGTB injection . . . 23
6.4 Hypotheses 4: 2 pass model architecture. . . . 23
6.4.1 not bothered to construct single pass architecture, requires building new mcts, fuck that . . . 23

7 evaluation . . . 24
7.1 fairness: same starting seed, epochs, search depth . . . 24
7.2 elo, benchmark a lot for good result . . . 24
7.3 illegal move probability . . . 24
7.4 policy entropy . . . 24
7.5 MCTS perft . . . 24
7.6 w/l to draw ratio . . . 24
7.7 graphs . . . 24

8 Conclusion . . . 25
8.1 what did i even do? i just fafo'd. . . . 25

9 challenges and areas of improvement . . . 26
9.1 building the chess client was challenging . . . 26
9.2 i shouldve used magic bitboards . . . 26
9.3 i shouldve adapted an existing chess engine . . . 26

9.4 i couldve trained and compared single vs dual pass transformer 26
9.5 trained for longer, whole point was long term but my hardware is bad 26
9.6 Using Typst 26
9.6.1 Adding figures 27
9.6.2 Adding tables 27
9.6.3 Adding equations 27
9.6.4 code blocks 28

Bibliography 29

List of Figures

Figure 1 An example figure, with two parts 27

List of Tables

Table 1 An example table, showing speed in operations per cycle per multiprocessor 26

Glossary

$MCTS$	Monte Carlo Search Tree, a heuristic search algorithm
$Policy$	The ith symbol in the source message, where $a_i \in S_m$
B_g	The decoded message, being a sequence of g source symbols $b_1 b_2 ... b_g$
b_i	The ith symbol in the decoded message, where $b_i \in S_m$
C_h	The transmitted (compressed) message, being a sequence of h Tunstall codewords $c_1 c_2 ... c_h$
c_i	The ith codeword in the transmitted message, where $c_i \in T_n$
D_h	The received (compressed) message, which for a complete Tunstall code is a sequence of h Tunstall codewords $d_1 d_2 ... d_h$
d_i	The ith codeword in the received message, where $d_i \in T_n$ for a complete Tunstall code

Abbreviations

MCTS	Monte Carlo Tree Search
BPSK	Binary Phase Shift Keying
BSC	Binary Symmetric Channel
DCT	Discrete Cosine Transform
ECC	Error Correcting Codes
FEC	Forward Error Correction
JPEG	Joint Photographic Experts Group
MPEG	Moving Pictures Experts Group
SER	Symbol Error Rate
SNR	Signal to Noise Ratio

Chapter 1

Introduction

1.1 Opening Statement

Machine learning is plagued with an acceptance of the ann as a black box. In my project, I attempt to deconstruct the learning process of transformer through a novel architecture, and by trying to develop emergent behaviour through wider and unrestricted action spaces in more humanistic action representations. the problem to be solved by the network will be chess, where there are highly ambiguous states with targets ranging from similarly ambiguous to objective ground truths. a problem that technically has a perfect solution, but impossible to find, seems most fitting for this investigation.

This project's focus is distributed over 4 hypothesis:

- whether self play in a pseudo legal chess environment converges to legal play.
- how masking logits affect learning dynamics, investigation into "grokking" mechanics
- the effectiveness of a 2 pass autoregressive encoder over a smaller action space in chess
- how the injection of perfect data at different stages of the training process influences agent strength

1.2 Aims of the Project

The aim of this project is to train multiple transformers on chess via self play reinforcement learning on legal and pseudo legal rule sets and investigate how the different action spaces influence agent performance over time to speculate on the impact in ideal conditions (e.g. with a lot more compute and time)

1.3 Objectives

- Build a bespoke chess client from scratch to handle both pseduo legal and legal game play, with:
  - a fucking move generator
  - zobrist hashing
  - state conversions to and from fen strings
  - bitboards and bitewise intrinsics
- Architect a 2 pass autoregressive encoder and integrate it with the chess engine
- construct a bipartite MCTS to self play on
- train on generated self play data

- collect statistics throughout training epochs, measuring leading statistics such as entropy, illegal move probability, elo
- from snapshots of model weights, estimate elo of engines
- evaluate impact of action space on agent
- analyse model weights to search for implicit masking
- suggest areas for improvement

1.4 Project Motivations

i like chess and ml and systems programming

1.5 Risks

there are a few risks with this project. one prominent risk is scope creep, especially with the bespoke construction of the chess client. in rust. that was just a terrible mistake, however i am just built different

Chapter 2

Literature Review

2.1 Chess fundamentals

2.1.1 Game rules

2.1.2 Board representation

2.1.3 Move generation

2.2 Machine learning in chess

2.2.1 Policy

2.2.2 Value

2.2.3 MCTS

2.3 History

2.3.1 Alpha beta pruning

2.3.2 NNUE

2.3.3 Transformers (attention is all you need)

2.4 Now

2.4.1 Grokking (a classic)

[1]

2.4.2 AlphaZero (Masking)

[2]

2.4.3 MuZero (Latent space)

[3] this destroys my disso, but i found out too late. maybe ill leave it out.

2.4.4 leela chess zero

[4]

2.4.5 Chess transformer

[5] not read, please be different to mine

2.4.6 Chessformer

[6] i also didnt know about this. neither have i read it, god i hope its no good.

Chapter 3

Research Hypotheses

3.1 Pseudo legal training

an agent learned to play pseudo legal chess should learn to play legal chess because king capture involves mating patterns and king safety involves pins

3.2 Punishment propogation

allowing illegal/geometrically impossible moves during self play and allowing the propogating the punishment back should teach the model just as well if not better in the long run

3.3 autoregressive move generator

more human like decision making approach in the action space is easier to train. trade off mcts depth vs width.

3.4 Perfect data injection

how does injection of perfect data throughout training influence agent? beginning: bootstrap value, forgetting. throughout: probably best at end: sharpens endgame, forgetting.

Chapter 4

Implementation details

the project is available on github.

4.1 System architecture overview

4.2 Client Architecture

4.2.1 Bitboard

4.2.2 Chessboard

4.2.3 ChessSquare

4.2.4 CastlingRights

4.2.5 ChessMove

4.2.6 ChessSquare

4.2.7 ChessGame

4.3 Model Architecture

4.3.1 Encoder

4.3.2 Positional Embeddings

4.3.3 Policy

4.3.4 Value

4.4 MCTS Architecture

4.4.1 Data

4.4.2 PUCT algorithm

4.4.3 Node

4.4.4 Edge

Chapter 5

Training & Self Play

5.1 Self play

5.2 MCTS

5.3 Training (backprop via autograd)

Chapter 6

Results

6.1 Hypotheses 1: pseudo legal

6.2 Hypotheses 2: masking

6.3 Hypotheses 2: EGTB injection

6.4 Hypotheses 4: 2 pass model architecture.

6.4.1 not bothered to construct single pass architecture, requires building new mcts, fuck that

Chapter 7

evaluation

7.1 fairness: same starting seed, epochs, search depth

7.2 elo, benchmark a lot for good result

7.3 illegal move probability

7.4 policy entropy

7.5 MCTS perft

7.6 w/l to draw ratio

7.7 graphs

Chapter 8

Conclusion

8.1 what did i even do? i just fafo'd.

Chapter 9

challenges and areas of improvement

9.1 building the chess client was challenging

9.2 i shouldve used magic bitboards

9.3 i shouldve adapted an existing chess engine

9.4 i couldve trained and compared single vs dual pass transformer

9.5 trained for longer, whole point was long term but my hardware is bad

9.6 Using Typst

Operation	Speed
Add, Mul, Mul-Add	8
Reciprocal	2
Divide	0.88
Divide Intrinsic	1.6
Recip. Square Root	2
Square Root	1
Logarithm	2
Exponent	1
Sin, Cos Intrinsics	2
Sin, Cos, Tan	Slow

Table 1: An example table, showing speed in operations per cycle per multiprocessor

9.6.1 Adding figures

Figures are added using the `#figure` function. Typst automatically handles the numbering and placement of your images, as shown in the example that produces Figure 1.1. Common formats like PNG, JPEG, and SVG are supported natively.

9.6.2 Adding tables

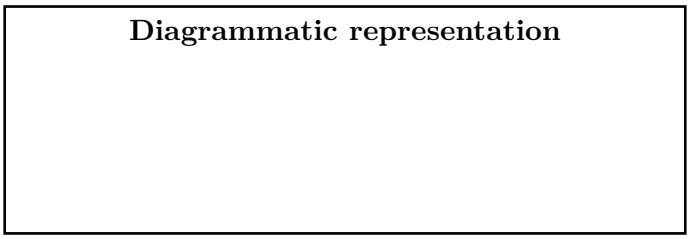
Tables are defined directly in the source code using the `#table` function. This provides a highly flexible way to grid your data. An example of a styled table is given in Table 1.1.

9.6.3 Adding equations

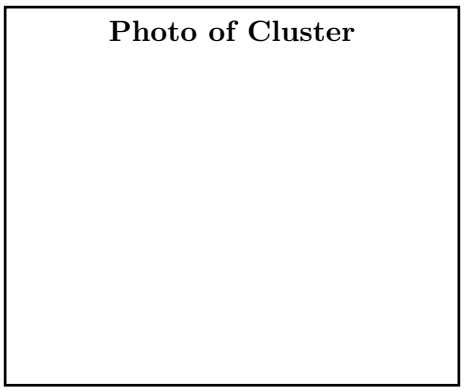
A primary advantage of Typst is its intuitive mathematical notation. Equations can be written within `$ ... $` delimiters. It handles numbered equations easily, as in the recursive formula:

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{ \boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1} \right\}$$

Inline math is also supported, for example to specify $1 \leq \iota < N$. Typst's syntax remains readable even for complex multiline expressions:



(a) Diagrammatic representation



(b) Photo of the Tempest cluster

Figure 1: An example figure, with two parts

$$\Pr\left\{ \boldsymbol{r}_{0, n\iota+x_2}, \sigma_{n\iota} = x_2 \right\} = \sum_{x_1, d_{\iota-1}} \left[\Pr\left\{ \boldsymbol{r}_{0, n(\iota-1)+x_1}, \sigma_{n(\iota-1)} = x_1 \right\} \right.$$
$$\left. \times \Pr\left\{ \boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1} \right\} \right]$$

9.6.4 code blocks

```
API:
pub struct ChessGame
pub ChessGame::castling_rights: CastlingRights
pub ChessGame::chessboard: ChessBoard
pub ChessGame::en_passant: Option<ChessSquare>
pub ChessGame::fullmove_counter: u32
pub ChessGame::game_history: alloc::vec::Vec<GameStateEntry>
pub ChessGame::halfmove_clock: u32
pub ChessGame::outcome: Outcome
pub ChessGame::rule_set: RuleSet
pub ChessGame::side_to_move: chess_piece::Color
pub ChessGame::zobrist_hash: u64
impl ChessGame
pub fn ChessGame::calculate_hash(&mut self) -> u64
pub fn ChessGame::check_game_state(&self) -> Outcome
pub fn ChessGame::fen_to_ascii(fen: &str)
pub fn ChessGame::from_fen(fen: &str) -> Self
pub fn ChessGame::generate_pseudolegal(&self) ->
alloc::vec::Vec<chess_move::ChessMove>
pub fn ChessGame::is_legal(&self, mov: &chess_move::ChessMove) -> bool
pub fn ChessGame::make_move(&mut self, mov: &chess_move::ChessMove)
pub fn ChessGame::to_fen(&self) -> alloc::string::String
pub fn ChessGame::uci_to_move(&self, input: &str) ->
result::Result<chess_move::ChessMove, &str>
```

Bibliography

[1] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177*, 2022.

[2] D. Silver *et al.*, "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm." [Online]. Available: https://arxiv.org/abs/1712.01815

[3] J. Schrittwieser *et al.*, "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, vol. 588, no. 7839, pp. 604–609, Dec. 2020, doi: 10.1038/s41586-020-03051-4.

[4] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons, and S. Russell, "Evidence of Learned Look-Ahead in a Chess-Playing Neural Network." [Online]. Available: https://arxiv.org/abs/2406.00877

[5] D. Monroe and P. A. Chalmers, "Mastering Chess with a Transformer Model." [Online]. Available: https://arxiv.org/abs/2409.12272

[6] D. Monroe, G. Eilender, P. Chalmers, Z. Tang, and A. Anderson, "Chessformer: A Unified Architecture for Chess Modeling," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=2ltBRzEHyd